

APPENDIX A INSTRUMENTATION OVERHEAD

In this experiment, we compare the efficiency of our path-aware instrumentation against the edge coverage one employed by `pcguard`. Table V shows the initial processing times of AFL++ as we provide it with a large queue from a 48-hour run and set it to execute each seed only once during calibration.

These time measurements include the costs of running the seeds (which are directly influenced by the instrumentation) and those required for ascertaining their coverage novelty. The values suggest that paths can be tracked with costs almost comparable to the edge coverage feedback of `pcguard`, as the geometric mean of the time ratios amounts to 1.26. Once a campaign starts, the fuzzer will also face mutation, scheduling, and other queue maintenance costs that make the direct impact (i.e., excluding queue explosion) of our instrumentation on fuzzing throughput even lower, as Section V-B discussed.

TABLE V
INPUT PROCESSING TIME (IN SECONDS) FOR A LARGE SET OF SEEDS: EDGE COVERAGE TRACKING (`PCGUARD`) VS. PATH-AWARE (`PATH`) INSTRUMENTATION.

Benchmark	pcguard	path	$\frac{\text{path}}{\text{pcguard}}$
cflow	8.39 s	9.56 s	1.14
exiv2	1.88 s	2.08 s	1.11
ffmpeg	2368.91 s	2216.81 s	0.94
flvmeta	0.16 s	0.29 s	1.81
gdk	17.18 s	17.8 s	1.04
imginfo	3.07 s	4.04 s	1.32
infotocap	410.96 s	504.99 s	1.23
jhead	0.31 s	0.56 s	1.81
jq	17.45 s	23.42 s	1.34
lame	733.9 s	740.16 s	1.01
mp3gain	20.68 s	25.42 s	1.23
mp42aac	7.49 s	9.52 s	1.27
mujs	13.68 s	17.99 s	1.32
nm-new	13.15 s	20.64 s	1.57
objdump	46.17 s	50.15 s	1.09
pdftotext	41.24 s	44.47 s	1.08
sqlite3	80.64 s	85.31 s	1.06
tiffsplit	7.89 s	14.68 s	1.86
GEOMEAN			1.26

APPENDIX B MEDIAN VALUES FOR FOUND BUGS

In the main evaluation, we followed the evaluation method of other works close to us in spirit (e.g., [4], [33]) that improve fuzzing sensitivity, comparing fuzzers for the bugs found within the campaign. We here report in Table VI the median number of bugs detected by each one of our fuzzers and `pcguard` across the 10 runs of the experiment we presented in Section V-A. The values in the table confirm the benefits of employing a path-aware feedback during fuzzing, as the trends we describe in Section V-A are mostly preserved (even though in some cases they can be slightly less evident).

APPENDIX C COMPARISON WITH PATHAFL

To experiment with PathAFL on the UNIFUZZ subjects, we followed the instructions³ that the authors provided for the running example in the publicly released artifacts. PathAFL uses `afl-gcc` to perform the compilation and initial instrumentation of the program under test, and then uses two scripts to extract information from the binary and patch it to complete the instrumentation. We made available gcc v. 9.4.0 and IDA Pro v. 7.0 to PathAFL to complete these tasks. Then, we tested the instrumented UNIFUZZ binaries for 48 hours in the custom PathAFL fuzzer, ensuring a fair and equal comparison with the results of our bug finding experiment from Section V-A.

Table VII shows the complete main results of the experiment. As we anticipated in Section V-A, PathAFL was only able to detect a fraction of the bugs brought to light by our fuzzers, corresponding to 29 of those discovered by our best-performing `cull` (29.5%), 30 of those found by the baseline `path` (38.9%) and 27 of those detected by `opp` (32.5%). However, PathAFL exposed 4 bugs (1 stack overflow on `exiv2`, 1 segmentation fault on `infotocap`, 1 heap overflow on `mujs`, and 1 exception-triggered abort on `pdftotext`) that none of our path-aware fuzzers brought to light, with `pcguard` missing them as well. All the other bugs found by PathAFL are also disclosed by at least one of our fuzzers. Furthermore, PathAFL found no bugs on 4 subjects, for which our path-aware fuzzers discovered more than 1 bug.

To investigate the reasons why our fuzzers missed those 4 bugs, we further tested the UNIFUZZ subjects using the baseline AFL fuzzer (v. 2.52b) atop which PathAFL was developed, in the same setup of the other experiments. Interestingly, AFL and PathAFL found nearly the same bugs: out of 34 bugs spotted by PathAFL and 32 by AFL, 31 were found by both. This means that PathAFL found only 3 bugs (2 in `objdump` and 1 in `infotocap`) that AFL was not able to detect, while it missed 1 (on `mp3gain`) found by its baseline. Notably, AFL finds 3 of the 4 bugs that both our path-aware fuzzers and `pcguard` missed, indicating that their discovery may not be dependent on the techniques introduced by PathAFL, but rather on the different underlying fuzzer. The `infotocap` one remains the only bug found in our tests solely by PathAFL.

To understand this modest performance of PathAFL, we observe that the evaluation from its paper revolves around “unique crashes” (and not unique bugs, unlike our work and more recent fuzzing evaluation practices). Reasoning on crashes inevitably leads to bug over-counting [32], as the PathAFL authors acknowledge in their paper [7]. Moreover, the authors employ the base AFL notion of “unique crashes”, which consists in considering any 2 crashes unique if they cover at least 1 different edge in the path from the beginning of the program to the crash site. This is an even more imprecise deduplication criterion, compared to the stack trace-based clustering [32] that we employed before manual analysis and

³https://github.com/yanxxd/PathAFL/blob/master/README_CN.md

TABLE VI
MEDIAN NUMBER OF UNIQUE BUGS FOUND BY EACH FUZZER CUMULATIVELY ACROSS THE 10 RUNS, FOLLOWED BY PAIRWISE COMPARISONS FOR COMMON (SET INTERSECTIONS) AND DIFFERENT (SET SUBTRACTIONS) BUGS.

Benchmark	path	pcguard	cull	opp	path $\cap$ pcguard	cull $\cap$ pcguard	opp $\cap$ pcguard	opp $\cap$ cull	path $\setminus$ pcguard	pcguard $\setminus$ path	cull $\setminus$ pcguard	pcguard $\setminus$ cull	opp $\setminus$ pcguard	pcguard $\setminus$ opp	opp $\setminus$ cull	cull $\setminus$ opp
cflow	2	1	2	1	1	1	1	1	1	0	1	0	0	0	0	1
exiv2	4	5	4	3	4	3	3	3	0	1	1	2	0	2	0	1
ffmpeg	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
flvmeta	2	2	2	2	2	2	2	2	0	0	0	0	0	0	0	0
gdk	6	6	8	6	6	6	6	6	0	0	2	0	0	0	0	2
imginfo	1	2	1	1	1	1	1	1	0	1	0	1	0	1	0	0
infotocap	1	4	1	3	1	1	3	1	0	3	0	3	0	1	2	0
jhead	6	5	6	5	5	5	5	5	1	0	1	0	0	0	0	1
jq	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
lame	4	4	4	3	4	4	3	3	0	0	0	0	0	1	1	1
mp3gain	3	3	3	3	2	2	2	2	1	1	1	1	1	1	1	1
mp42aac	6	5	6	5	5	5	4	4	1	0	1	1	1	1	2	2
mujs	3	2	2	1	2	2	1	1	1	0	0	0	0	1	0	1
nm-new	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
objdump	9	8	11	9	8	8	7	9	1	0	3	0	2	1	2	2
pdftotext	8	9	16	8	5	7	7	8	3	4	9	2	1	2	0	8
sqlite3	5	8	6	6	4	6	4	4	1	4	0	2	2	4	2	2
tiffsplit	5	5	6	4	4	5	4	4	1	1	1	0	0	1	2	2
TOTAL	48	55	58	47	41	42	45	42	7	14	16	13	2	10	5	16

TABLE VII
UNIQUE BUGS FOUND BY OUR PATH-AWARE FUZZERS AND PATHAFL, CUMULATIVELY ACROSS THE 10 RUNS, FOLLOWED BY PAIRWISE COMPARISONS FOR COMMON (SET INTERSECTIONS) AND DIFFERENT (SET SUBTRACTIONS) BUGS.

Benchmark	path	pathafl	cull	opp	path $\cap$ pathafl	cull $\cap$ pathafl	opp $\cap$ pathafl	path $\setminus$ pathafl	pathafl $\setminus$ path	cull $\setminus$ pathafl	pathafl $\setminus$ cull	opp $\setminus$ pathafl	pathafl $\setminus$ opp
cflow	2	0	3	2	0	0	0	2	0	3	0	2	0
exiv2	8	4	6	8	3	3	3	5	1	3	1	5	1
ffmpeg	2	2	1	0	2	1	0	0	0	0	1	0	2
flvmeta	2	2	2	2	2	2	2	0	0	0	0	0	0
gdk	8	0	11	9	0	0	0	8	0	11	0	9	0
imginfo	2	0	2	3	0	0	0	2	0	2	0	3	0
infotocap	2	3	3	5	2	2	2	0	0	1	0	3	0
jhead	6	6	6	6	6	6	6	0	0	0	0	0	0
jq	1	1	1	1	1	1	1	0	0	0	0	0	0
lame	4	3	5	4	3	3	3	1	0	2	0	1	0
mp3gain	3	1	3	3	1	1	1	2	0	2	0	2	0
mp42aac	7	0	8	6	0	0	0	7	0	8	0	6	0
mujs	3	1	4	1	0	0	0	3	1	4	1	1	1
nm-new	0	0	0	0	0	0	0	0	0	0	0	0	0
objdump	9	5	12	11	5	5	5	4	0	7	0	6	0
pdftotext	8	3	18	10	2	2	2	6	1	16	1	8	1
sqlite3	5	1	7	7	1	1	1	4	0	6	0	6	0
tiffsplit	5	4	6	5	4	4	3	1	0	2	0	2	1
TOTAL	77	34	98	83	30	29	27	47	4	69	5	56	7

that much literature used as evaluation practice before moving, more recently, to manually deduplicated unique bugs.

We provide in Table IX a comparison of the crashes detected by AFL and PathAFL, using both their notion of “unique crashes” (*crashes* columns 2 and 4) and the more standard one we adopted for our main experiment in Section V-A (*unique crashes* columns 3 and 5), i.e., the hashing-based clustering that considers the top 5 frames of the stack trace leading to the crash. The results of this analysis indicate that PathAFL actually finds a considerable number of crashes that are missed by the base AFL, which is in line with what the authors claim in their paper, but in the end most of these crashes are actually different manifestations of the same bugs.

In light of the results of this experiment, we conclude that our methods evidently and largely **outperform** the one proposed

by Shengbo et al. [7], as they proved capable of discovering many more, and different, bugs that neither PathAFL nor our pcguard baseline were able to spot.

APPENDIX D CULLING WITH RANDOM SELECTION

To complete our investigation into the source of the experimental differences between our approach and classic edge coverage-guided fuzzing, we conducted an ablation study by replacing the edge coverage-preserving criterion for culling with a random selection strategy. The rationale behind this experiment is to understand whether retaining edge coverage is the key driver of the benefits of our culling technique, and whether reducing the queue size already improves the results of the baseline path-aware fuzzer.

TABLE VIII
UNIQUE BUGS FOUND BY PATHAFL AND AFL CUMULATIVELY ACROSS THE 10 RUNS, FOLLOWED BY PAIRWISE COMPARISONS FOR COMMON (SET INTERSECTIONS) AND DIFFERENT (SET SUBTRACTIONS) BUGS.

Benchmark	pathafl	afl	pathafl $\cap$ afl	pathafl $\setminus$ afl	afl $\setminus$ pathafl
cflow	0	0	0	0	0
exiv2	4	4	4	0	0
ffmpeg	2	2	2	0	0
flvmeta	1	1	1	0	0
gdk	0	0	0	0	0
imginfo	0	0	0	0	0
infotocap	3	2	2	1	0
jhead	6	6	6	0	0
jq	1	1	1	0	0
lame	2	2	2	0	0
mp3gain	1	2	1	0	1
mp42aac	0	0	0	0	0
mujs	1	1	1	0	0
nm-new	0	0	0	0	0
objdump	5	3	3	2	0
pdftotext	3	3	3	0	0
sqlite3	1	1	1	0	0
tiffsplit	4	4	4	0	0
TOTAL	34	32	31	3	1

TABLE IX
CRASHES AND UNIQUE CRASHES FOUND BY PATHAFL AND AFL CUMULATIVELY ACROSS THE 10 RUNS.

Benchmark	pathafl		afl	
	crashes	unique crashes	crashes	unique crashes
cflow	0	0	0	0
exiv2	103	20	72	14
ffmpeg	7	2	7	2
flvmeta	747	1	917	1
gdk	0	0	0	0
imginfo	0	0	0	0
infotocap	304	3	980	3
jhead	311	9	296	11
jq	31	3	107	3
lame	11	4	17	5
mp3gain	472	2	705	3
mp42aac	0	0	0	0
mujs	442	2	1609	2
nm-new	0	0	0	0
objdump	56	11	22	4
pdftotext	31	3	35	4
sqlite3	2	1	4	1
tiffsplit	2341	23	1768	21
TOTAL	4858	84	6539	74

TABLE X
UNIQUE BUGS FOUND BY THE PATH AND CULL FUZZERS AND THE CULL_R VARIANT FOR THE ABLATION EXPERIMENT, CUMULATIVELY ACROSS THE 10 RUNS, FOLLOWED BY PAIRWISE COMPARISONS FOR COMMON (SET INTERSECTIONS) AND DIFFERENT (SET SUBTRACTIONS) BUGS.

Benchmark	path	cull _r	cull	path $\cap$ cull _r	cull $\cap$ cull _r	path $\setminus$ cull _r	cull _r $\setminus$ path	cull $\setminus$ cull _r	cull _r $\setminus$ cull
cflow	2	2	3	2	2	0	0	1	0
exiv2	8	9	6	8	6	0	1	0	3
ffmpeg	2	1	1	1	1	1	0	0	0
flvmeta	2	1	2	1	1	1	0	1	0
gdk	8	10	11	8	10	0	2	1	0
imginfo	2	2	2	2	2	0	0	0	0
infotocap	2	2	3	2	2	0	0	1	0
jhead	6	6	6	6	6	0	0	0	0
jq	1	1	1	1	1	0	0	0	0
lame	4	4	5	4	4	0	0	1	0
mp3gain	3	4	3	3	3	2	3	0	1
mp42aac	7	7	8	7	6	0	0	2	1
mujs	3	2	4	1	1	2	1	3	1
nm-new	0	0	0	0	0	0	0	0	0
objdump	9	11	12	7	11	2	4	1	0
pdftotext	8	10	18	7	7	1	3	11	3
sqlite3	5	3	7	2	2	3	1	5	1
tiffsplit	5	6	6	4	5	1	2	1	1
TOTAL	77	81	98	64	70	13	17	28	11

For the ablation, we first analyzed the average size of the queue after each culling round performed by the `cull` fuzzer for each subject, and used those values as range of options to decide the quantity of test cases to randomly remove at each culling round. For the removal, we seed the random number generator with the timestamp of each culling round, and trim from the queue between 84% and 98% of its elements.

We show in Table X the outcome of this experiment, where we compare the unique bugs discovered by this new fuzzer configuration, which we name `cullr`, against those discovered by `path` and `cull` after 48 hours of testing with 10 runs per subject, as we did for all the main experiments.

As the reader can appreciate, `cullr` discovered 4 more bugs (81 in total) than the baseline `path` (77 in total), and 17 fewer bugs than `cull` (98 in total). This indicates that an unoptimized culling strategy may already bring an improvement with respect to the baseline path-aware fuzzer, which is overburdened by the queue explosion phenomenon. On the other hand, it is also evident that just removing test cases randomly is inefficient for the total number of bugs found when compared to `cull`, as an inevitable outcome of this operation is causing regression and thus requiring rediscovery of certain code areas.

Interestingly, `cullr` was able to disclose 17 and 11 bugs missed by `path` and `cull`, respectively. Further analysis shows that `cullr` exposed 5 bugs that no other path-aware fuzzer (`path`, `cull`, `opp`) could disclose. These bugs affect `mp3gain`, `mujs`, `pdftotext`, `sqlite3`, and `tiffplit` (one each per program), and `pcguard` could expose only the one affecting `sqlite3`. These results further substantiate our claim that there is no single right way of wielding the increased visibility brought about by our path-aware feedback, warranting future exploration of this research direction.

For a comparison with the remaining path-aware fuzzer `opp`, we observe that `cullr` slightly underperforms in terms of total number of disclosed bugs: 83 for `opp` and 81 for `cullr`. However, when taking into account bug identity, `cullr` was capable of discovering 13 bugs missed by `opp`, whereas `opp` found 15 bugs missed by `cullr`. This indicates that random culling and the opportunistic biasing approach have their own advantages and neither strategy supersedes the other.